

Git & GitHub Reference Cheat Sheet

1. Command Line Basics

```
ls
```

List files and folders.

```
cd Desktop/folder
```

Go into a folder.

Remember: Run Git commands inside your project folder.

2. Creating Commits

A **commit** is a saved version of your code.

```
git init
```

Start Git in the current folder.

```
git status
```

See what changed.

```
git add .
```

Prepare all changed files.

```
git commit -m "message"
```

Save the changes as a commit.

```
git log
```

See commit history.

```
git log --all --graph
```

See branches and commits visually.

2. Creating Commits — More

Set your name and email once:

```
git config --global user.name "Your Name"
```

```
git config --global user.email "email@example.com"
```

```
git commit -m "message" --amend
```

Change the previous commit.

3. The Three Areas in Git

Area	Simple meaning
Working Area	Your actual files and changes
Staging Area	Changes selected for the next commit
Commit History	Saved versions

```
git add . # working → staging
git commit -m "message" # staging → saved commit
git reset file # unstage file
git reset . # unstage everything
git checkout -- file # remove changes from file
git checkout -- . # remove all current changes
```

4. Viewing & Restoring Previous Commits

```
git checkout abc123
```

View an older commit using its hash.

```
git checkout main
```

Go back to the main branch.

```
git checkout abc123 file
```

Restore one file from an older commit.

```
git checkout abc123 folder/
```

Restore a folder from an older commit.

```
git checkout abc123 .
```

Restore the whole project from an older commit.

HEAD tells you where you are currently viewing.

5. Other Useful Git Features

```
git config --global alias.s "status"
```

Create a shortcut: git s = git status.

```
rm -rf .git
```

Remove Git tracking from the project.

.gitignore tells Git which files/folders not to track, such as node_modules/ and .env.

6. GitHub Basics

Repository: Project folder tracked by Git.

GitHub: Online place to store Git repositories.

Local repository: Git repository on your computer.

Remote repository: Git repository online, such as GitHub.

Alternatives: Bitbucket, GitLab.

7. Uploading Code to GitHub

```
git remote add origin URL
```

Connect your local project to GitHub.

```
git remote
```

See connected remotes.

```
git remote -v
```

See the remote URLs.

```
git remote remove origin
```

Remove the GitHub connection.

```
git push origin main
```

Upload your main branch.

```
git push origin main --set-upstream
```

Connect main so later you can simply use git push.

```
git push origin main -f
```

Force-push and overwrite remote history.

8. Downloading Code from GitHub

```
git clone URL
```

Download a GitHub repository.

```
git clone URL folder
```

Download it into a chosen folder.

```
git fetch
```

Get information about remote changes.

```
git pull origin main
```

Download and merge the latest main branch.

```
git pull origin main --set-upstream
```

Set a shortcut for later git pull.

9. Branching

A branch lets you work on a feature without changing the main branch.

```
git branch feature1
```

Create a branch.

```
git checkout feature1
```

Switch to the branch.

```
git branch -D feature1
```

Delete the branch.

HEAD → **feature1** means you are currently on feature1.

10. Merging

```
git checkout main
```

Go to the branch that will receive the changes.

```
git merge feature1 -m "message"
```

Merge feature1 into main.

10. Merge Conflicts

A conflict can happen when two branches change the same line.

```
<<<<<< HEAD
code from your current branch
=====
code from the other branch
>>>>>> feature1
```

Fix: Delete the conflict markers and keep the code you want.

```
git add .
git commit -m "message"
```

11. Feature Branch Workflow

1. Create and switch to a feature branch

```
git branch new-feature  
git checkout new-feature  
# make your changes  
git add .  
git commit -m "new feature"
```

2. Upload the feature branch

```
git push origin new-feature
```

3. Create a Pull Request on GitHub

Team members can review the code and leave comments.

4. Merge the Pull Request into main

Open the Pull Request and click Merge.

5. Update your local main

```
git checkout main  
git pull origin main
```

12. Merge Conflicts in Feature Branch Workflow

If two Pull Requests change the same file on the same line, a conflict may occur.

Option A — GitHub: Open the Pull Request → Resolve conflicts → edit the file.

Option B — Computer:

```
git checkout main
git pull origin main
git checkout feature-branch
git pull origin feature-branch
git merge main
# fix the conflict
git add .
git commit -m "message"
git push origin feature-branch
```

After this, the Pull Request should be ready to merge.

13. Quick Reference

```
git init Start Git
git status See changes
git add . Stage changes
git commit -m "msg" Save version
git log See history
git branch name Create branch
git checkout name Switch branch
git merge name -m "msg" Merge branch
git push origin main Upload
git pull origin main Download + merge
git clone URL Download repository
git fetch Get remote updates
git reset file Unstage file
git remote -v See remote URL
```